https://doi.org/10.26599/IJCS.2023.9100018 International Journal of Crowd Science

Development and Investigation of Vision System for a Small-Sized Mobile Humanoid Robot in a Smart Environment

Amer Tahseen Abu-Jassar[1], Hani Attar[2, 3] ✉, Ayman Amer[2], Vyacheslav Lyashenko[4], Vladyslav Yevsieiev[4], and Ahmed Solyman[5]

ABSTRACT

The conducted research aims to develop a computer vision system for a small-sized mobile humanoid robot. The decentralization of the servomotor control and the computer vision systems is investigated based on the hardware solution point of view, moreover, the required software level to achieve an efficient matched design is obtained. A computer vision system using the upgraded tiny-You Only Look Once (YOLO) network model is developed to allow recognizing and identifying objects and making decisions on interacting with them, which is recommended for crowd environment. During the research, a concept of a computer vision system was developed, which describes the interaction between the main elements, on the basis of which hardware modules were selected to implement the task. A structure of information interaction between hardware modules is proposed, and a connection scheme is developed, on the basis of which a model of a computer vision system is assembled for research, with the required algorithmic and software for solving the problem. To ensure the high speed of the computer vision system based on the ESP32-CAM module, the neural network was improved by replacing the Visual Geometry Group 16 (VGG-16) network as the base network for extracting the functions of the Single Shot Detector (SSD) network model with the tiny-YOLO lightweight network model, which made it possible to preserve the multidimensional structure of the network model feature graph, resulting in increasing the detection accuracy, while significantly reducing the amount of calculations generated by the network operation, thereby significantly increasing the detection speed, due to a limited set of objects. Finally, a number of experiments were carried out, both in static and dynamic environments, which showed a high accuracy of identifications.

KEYWORDS

social system; assistant robotics; autonomous robots; humanoid robot; computer vision system; neural networks; decision making; crowd environment

The evolution of modern robotics development is based on new technologies introduction in the field of artificial intelligence, decision making, processing of large amounts of data, and visual information[1–6]. One of the basic requirements for mobile robots is the availability of a computer vision system, which makes it possible to transmit the state of the environment to the operator in real time[7–9]. This allows the operator to assess the situation and complete the assigned tasks. However, some solutions in this area limit the autonomy of the mobile robot and make it dependent on the presence of the operator. To ensure the autonomy of such robots, it is necessary not just to implement a system for broadcasting a video stream of the environment of a mobile robot, but to develop a system for identifying objects in real time, with the ability to further use the data obtained at the object to develop a strategy for behavior and decision making.

In Ref. [10], Bavelos et al. proposed the development of a computer vision system for an industrial mobile robot that can autonomously move around the workshop. The peculiarity of the proposed solution lies in the synthesis of data from sensors of a mobile robot with 2D and 3D sensors located in the workshop. This, according to the authors, allows the mobile robot to perceive the ongoing processes around it in real time[10]. This solution makes it possible to more accurately describe the events that occur in the working area of a mobile robot, but the proposed method cannot be used in open areas, which ties it to a specific location.

Al-Obaidi et al.[11] conducted research to develop a mobile robot for remote security monitoring in factories, offices, and airports. The main goal of this study, according to the authors, is the development of an energy-saving mobile robot with a computer vision system and environmental control due to an array of sensors (temperature and presence of obstacles). To implement a computer vision system, Al-Obaidi et al.[11] used a Raspberry Pi with a connected Camera Board, which made it possible to broadcast a video stream from a mobile robot to a control system, and an ATMEGA 328P microcontroller based on Arduino was used as a motor control system. Considering the proposed solution, it can be seen that the use of Raspberry Pi as a computer vision system and Arduino as a motor control system is rational. But the presence of specific power requirements for the Raspberry Pi (DC 5 V, 1 A) failure to comply with these recommendations can lead to unstable operation of the mobile robot, and therefore, if Al-Obaidi et al.[11] expanded the functionality of the computer

1 Department of Computer Science, College of Information Technology, Amman Arab University, Amman 11937, Jordan
2 Faculty of Engineering, Zarqa University, Zarqa 2000, Jordan
3 College of Engineering, University of Business and Technology, Jeddah 21448, Saudi Arabia
4 Department of Media Systems and Technology, Kharkiv National University of Radio Electronics, Kharkiv 61166, Ukraine
5 Department of Electrical and Electronics Engineering, Faculty of Engineering and Architecture, Nişantaşı University, Istanbul 34398, Türkiye
Address correspondence to Hani Attar, Hattar@zu.edu.jo

© The author(s) 2025. The articles published in this open access journal are distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/).

vision system by introducing neural networks for object recognition and decision-making systems, will entail either an unstable robot or the need to completely change the power supply system of the mobile robot.

In Ref. [12], an example of the development of a multi-sensor recognition system based on Light Detection and Ranging (LiDAR) for the mobile robot iRobotic Packbot 501 was given. The proposed solution allows, in synthesis with a computer vision system using LiDAR, to build a 3D map of the environment[12]. But for the implementation of such a system, it is necessary to use OS Robot Operating System (ROS), which requires high-performance hardware based on expensive microcomputers and will correspondingly increase the cost and impossibility of using it on small-sized robots.

Table 1 compares some of the existing methods for obtaining visual information about mobile robot environment, on the basis of which an intelligent decision-making system can be developed.

As can be seen from Table 1, the following methods were chosen for analysis:

2D imaging (camera) can track moving objects in real time and locate them in the straight sight of the mobile robot. This allows to detect objects in dynamic space and makes it possible to determine their location and identification using Artificial Intelligence (AI).

3D vision is a complex method of work synthesis with two cameras or a laser scanner located at different angles. As a result, there are high requirements for hardware and software implementation to visualize the surrounding space of a mobile robot. But such a system has a high accuracy of recognition of the objects presence, without the possibility of their identification, and has a small delay in real time.

The ultrasonic method consists of a device that calculates the time interval between emission and detection of a reflected sound wave, which allows to determine the presence of objectives and the distance to the sensor, and hence, makes a decision based on the results. The ultrasonic device works in real time and it is easy to be implemented and programmed. However, it does not allow the visualization of the environment resulting in failing on identifying objects, which means that it only recognizes the presence of the sensors nearby objectives.

The infrared method detects Infrared (IR) rays emitted by an object. It can also use IR light to project onto a target and receive reflected light to determine its distance or proximity. Infrared sensors are economical and can track infrared light over a large area. It also work in real time. It allows to determine the presence of an object, but does not allow to identify it, and is simple enough to implement both software and hardware.

Thus, to solve the problem of developing a computer vision system for small-sized robots with the possibility of implementing an AI-based identification system, we choose to use the 2D imaging method. To conduct experimental research in the role of an actuator and work out interaction with the outside world, a prototype of a humanoid robot will be used, which will expand the possibility of implementing manipulation capabilities for interacting with objects and make it possible in the future to implement a group control and decision-making system to achieve the set tasks.

Thus, in this study, the authors set themselves the task of developing a computer vision system using AI technologies to recognize and identify objects in the robot's work area. At the same time, strict requirements are put forward for the overall dimensions and computing power of the hardware. This makes it possible to use it inside small-sized intelligent humanoid robots, as well as robots for rescue missions in areas of man-made disasters. At this point in time, existing similar solutions use expensive hardware such as Nvidia Jetson Orin Nano or Raspberry Pi 4 Model B 4GB. In addition, the listed hardware has increased requirements for the power circuit, as well as large overall dimensions, which do not meet the requirements for a small-sized humanoid robot.

1 General Concept of Mobile Robot Computer Vision Control System

Analyzing the specifics and complexity of the developed mobile humanoid robot, the authors decided to apply a decentralized approach to the control system by dividing the humanoid robot motion control system and the computer vision system into separate modules. Grounding for this decision was next: the bandwidth of the video channel increases, which reduces the signal delay time in the computer vision system; and the load on the resolution capabilities of the microcontroller module of the servomotor control system is reduced, which speeds up the execution of commands. All actions for recognition and identification using AI, as well as algorithms for making decisions and interacting with the outside world and objects, are implemented on a personal computer (notebook).

At the first step in solving the problem, it is necessary to develop a general concept for the control system implementation, which should include a computer vision module and will be used to transfer the video stream to the laptop and return commands to interact with objects (ball). Figure 1 shows the general concept for the control system implementation for a mobile humanoid robot.

Table 1 Methods for obtaining visual information about mobile robot environment comparison.

Parameter	2D imaging (camera)[13]	3D vision (LiDAR)[14]	Ultrasonic method (sensor)[15]	Infrared method (sensor)[16]
Recognition accuracy	++	+++	+	–
Resolution	++	+++	–	–
"Dead" zone	+	–	++	++
Field of View (FOV)	++	+++	+	+
Dimension	+	+++	+	+
Price	+	+++	+	+
Hardware requirements	++	+++	+	+
Software implementation complexity	++	+++	+	+
Object identification	+++	++	–	–

Note: +++: high rates; ++: middle rates: +: low rates; –: impossible to implement.

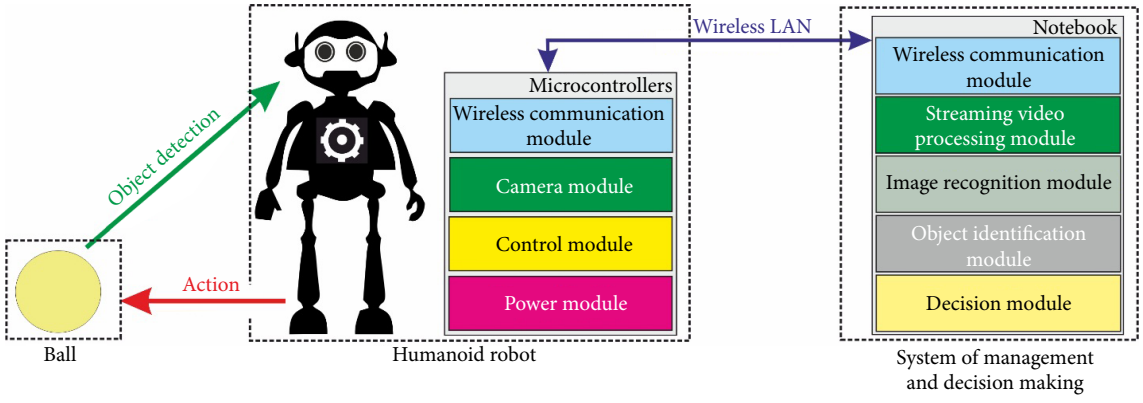


Fig. 1 General concept for the control system implementation for a mobile robot.

The basic idea of this concept is the development of a decentralized control system, that is, its division into two subsystems. Very strong restrictions on the overall dimensions of the humanoid robot became grounding for this decision. As a result, four main modules will be placed on the basis of the humanoid robot: wireless communication module; camera module; control module; and power module.

Wireless communication module allows to exchange data between a humanoid robot and a control system, decision-making using wireless Local Area Network (LAN) technologies[17].

Camera module will perform the function of broadcasting the mobile robot environment using wireless network technologies.

Control module is designed to execute commands that the module receives from the control and decision-making system, which is located on a personal computer (notebook). An additional task of the control module is to implement the ability to connect servomotors, which will provide freedom of action for the humanoid robot to perform the tasks.

Power module provides a small-sized battery assembly, which will be used to power a humanoid robot.

On the basis of a personal computer (notebook), it is proposed to create a control and decision-making system, and this decision is justified by the fact that the development of an object recognition and identification system, as well as a decision-making system, requires "serious" computing power that cannot be placed on the basis of a humanoid robot. As a result, this subsystem consists of the following modules: wireless communication module; streaming video processing module; image recognition module; object identification module; and decision module.

Streaming video processing module is designed for primary processing of streaming video received from a humanoid robot and preparing it for the recognition module.

Image recognition module processes the received streaming video, divides it into frames, and recognizes the presence of objects in the frame.

The object identification module uses the neural networks to identify the robot's nearby objectives, i.e., the objectives located in the robot's working zone. Consequently, the object identification module gives the suggested name of the identified object and the accuracy of the suggestion.

Decision module analyzes the results obtained with the object identification module and, depending on the underlying algorithm, performs interaction actions with the object.

2 Selection of Hardware Modules for the Robot's Computer Vision Control System

The humanoid mobile robot "ViVi" by Doctors of Intelligence and Technology LTD is used as a model for conducting research. Overall dimensions are: height is 200 mm; width is 72.2 mm; arm length (from shoulder to hand) is 105 mm; movement provides 18 servomotors model EMax ES08MAII.

Based on the concept proposed above for a computer vision system for a humanoid robot (Fig. 1), at the beginning of development, it is necessary to discuss the restrictions that are imposed when choosing hardware modules, on the basis of which the camera module and control module will be implemented.

Dimension: Camera module is no more than 30 mm × 45 mm × 15 mm (width × length × height). Control module is no more than 54 mm × 51 mm × 7 mm (width × length × height).

Power: Camera module working voltage is 5 V, and average current is 180 mA (max); Control module working voltage is 4.5–6.4 V, and average current is 80 mA (min).

Wireless communication module: WiFi 802.11 b/g/n support for camera module and control module.

Required camera module features are as follows:

(1) Connection of a camera with the ability to transmit streaming video with a minimum resolution of 800 pixel × 600 pixel 30 frames per second (fps);

(2) The ability to create an access point to a wireless network based on the module.

Required control module functions are as follows:

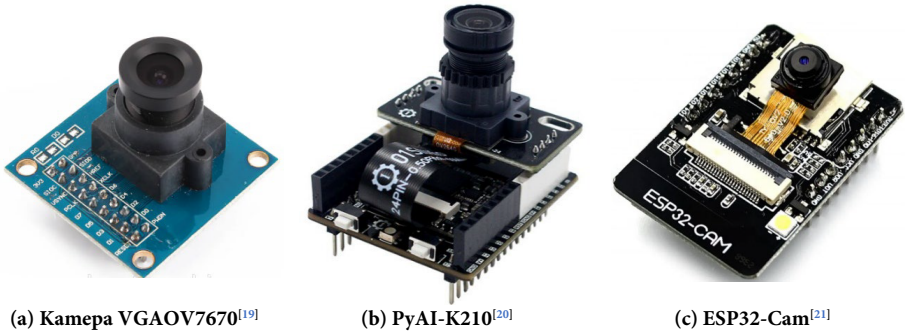
(1) Power supply support for up to 18 Emax ES08MA II servomotors[18];

(2) The maximum average current (when 18 servomotors are working) can reach 2800 mA.

The rationale for the selected restrictions and their values are the overall dimensions of the humanoid robot: total 72 mm × 200 mm × 80 mm (width × length × height) and body dimensions 40 mm × 55 mm × 80 mm (width × length × height), which should accommodate control module, camera module, and power module. As a result, the developed solution has a rigid framework for the analysis and selection of hardware modules.

First, we will analyze and select a hardware module for the implementation of the camera module, and based on the above limitations, the following modules were selected, which are shown in Fig. 2, and their parameters are presented in Table 2.

Having carried out a comparative analysis of the parameters of hardware modules (Table 2) for the implementation of a computer vision system for a humanoid robot, we can identify the following advantages and disadvantages for solving the problem. The OV7670 VGA hardware module fits the overall dimensions and has an attractive price (5 US dollar). However, it has a number of disadvantages, such as the absence of a wireless network module and a microcontroller-based module for primary processing of the video stream; and to use this module, you need



(a) Kamepa VGAOV7670[19] (b) PyAI-K210[20] (c) ESP32-Cam[21]

Fig. 2 Hardware modules for computer vision system implementation.

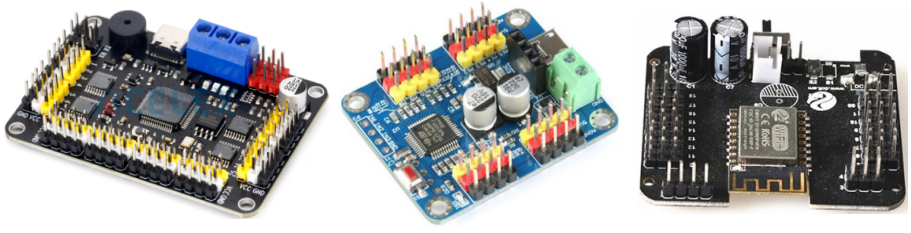
Table 2 Comparison of main parameters of camera module.

Parameter	VGA OV7670[22]	PyAI-K210[23]	ESP32-Cam[24]
Microcontroller	Additional module	ESP32 (240 MHz)	ESP32 (240 MHz)
Wireless communication module	Additional module	+ (Wi-Fi)	+ (Wi-Fi)
Maximum resolution	640 pixel×480 pixel (0.3 megapixel)	UXGA 1600 pixel×1200 pixel (2 megapixel)	UXGA 1600 pixel×1200 pixel (2 megapixel)
Format support	VGA, QVGA, CIF, and QCIF	UXGA,SXG, SVGA, QVGA, CIF, and QCIF	UXGA,SXG, SVGA, QVGA, CIF, and QCIF
Video streaming rate (fps)	VGA: 30	UXGA/SXGA: 15–30, SVGA: 30, and CIF: 60	UXGA/SXGA: 15–30, SVGA: 30, and CIF: 60
Available color code	Raw RGB, RGB (GRB 4:2:2, RGB565/555/444), YUV (4:2:2), and YCbCr (4:2:2)	YUV(422/ 420)/YCnCr422, RGB565/555, and 8-bit compressed data	YUV(422/ 420)/YCnCr422, RGB565/555, and 8-bit compressed data
Voltage (V)	2.5–3.0	3.6–6.0	5.0
Board dimension (mm×mm)	35×35	38×42	27×39
Weight (g)	8	18	10
Price (US dollar)	5	50	8–10

to purchase separately a Wi-Fi module and a control module (Arduino Min, STM32), which accordingly increases not only the cost but also the complexity of the design, as well as overall dimensions. At the same time, the speed of the microcontroller will be significantly lower (16–32 MHz) than that of the pyAI-K210 and ESP32 modules. Comparing the pyAI-K210 and ESP32 hardware modules, it can be seen that these two modules have similar parameters in terms of microcontroller speed and camera characteristics. The main differences between such modules are the overall dimensions and weight of the hardware module, but the price of ESP32-Cam is about 4.5–5.0 times lower than pyAI-K210 and has a wider distribution and software support for libraries for working with modules and sensors. Based on this comparison, an ESP32-Cam will be used to implement the camera module onboard a humanoid robot.

The next step is to select a hardware module for the implementation of the control module by a humanoid robot. Based on the above restrictions, the following hardware modules fell into the analysis area, which are presented in Fig. 3, and their parameters are presented in Table 3.

Analyzing the main parameters of the control system hardware modules, it can be seen that the 16 road steering gear control board servo and neue version 32 Kanal roboter servo control board modules are based on the STM 32 bit microcontroller. At the same time, the neue version 32 Kanal roboter servo control board has a module for wireless data transmission via Wi-Fi and bluetooth, which fits certain restrictions, but these modules do not fit in terms of overall dimensions, as well as the number of servomotors that can be connected to the board management. On the other hand, the control system hardware of neue version 32



(a) Neue version 32 Kanal roboter servo control board[25] (b) 16 road steering gear control board servo[26] (c) Humanoiden roboter control board[27]

Fig. 3 Hardware modules for the humanoid robot motion control system implementation.

Table 3 Comparison of the hardware modules' main parameters for the control system development.

Parameter	Neue version 32 Kanal roboter servo control board[25]	16 road steering gear control board servo[26]	Humanoiden roboter control board[27]
CPU	STM 32 bit	STM 32 bit	ESP8266MOD
Flash capacity (Mbit)	16	16	4
Wireless communication	Wi-Fi and bluetooth	–	Wi-Fi
Servomotor controller the number of simultaneous	32	16	18
Servomotor input voltage (V)	4.2–7.2	4.2–7.2	4.2–7.2
Operation voltage (V)	5	5	3.3
Communication protocol	UART	USB-TTL	TTL programmer
Board dimension (mm×mm)	45×63	36×43.5	24×16
Weight (g)	8	12	12
Price (US dollar)	20	5	15

Kanal roboter servo control board, 32-Emax ES08MA II servomotor, and 16-road steering gear control board servo, will not allow to realize all degrees of freedom for a humanoid robot. Considering the parameters of the humanoiden roboter control board, it can be seen that it not only is suitable in terms of overall dimensions, but also is determined by the presence of a Wi-Fi module onboard a control system based on the ESP8266MOD microcontroller. This will make it possible to create an access point on its basis and simplify the process of transmitting commands to a humanoid robot from a control and decision-making system; as a result, to implement a control system for a humanoid robot, the authors suggest using the humanoiden roboter control board for 15 US dollar.

3 Structural Diagram of Interaction Between Main Elements of Mobile Robot Computer Vision Control System

After choosing the hardware modules, it is necessary to develop a general structure for the information interaction of the camera module and control module with the system of management and decision making to control the humanoid robot. This structure is shown in Fig. 4.

Based on the developed structure of information interaction between software and hardware modules, taking into account the specifics of developing programs for a microcontroller, to implement a computer vision system for a mobile humanoid robot based on ESP32-Cam and a neural network for object recognition and identification, it is proposed to use the following set of libraries supported in the environment Arduino IDE development[28]:

WebServer makes it possible to implement a simple web server with support for only one client at a time, which allows to process requests using the Get and Post methods. The built-in IoT Development Framework from Espressif Systems (ESP-IDF) web server will be used. This web server is based on the Transmission Control Protocol/Internet Protocol (TCP/IP) stack and allows developers to create and conFig. web interfaces to manage and

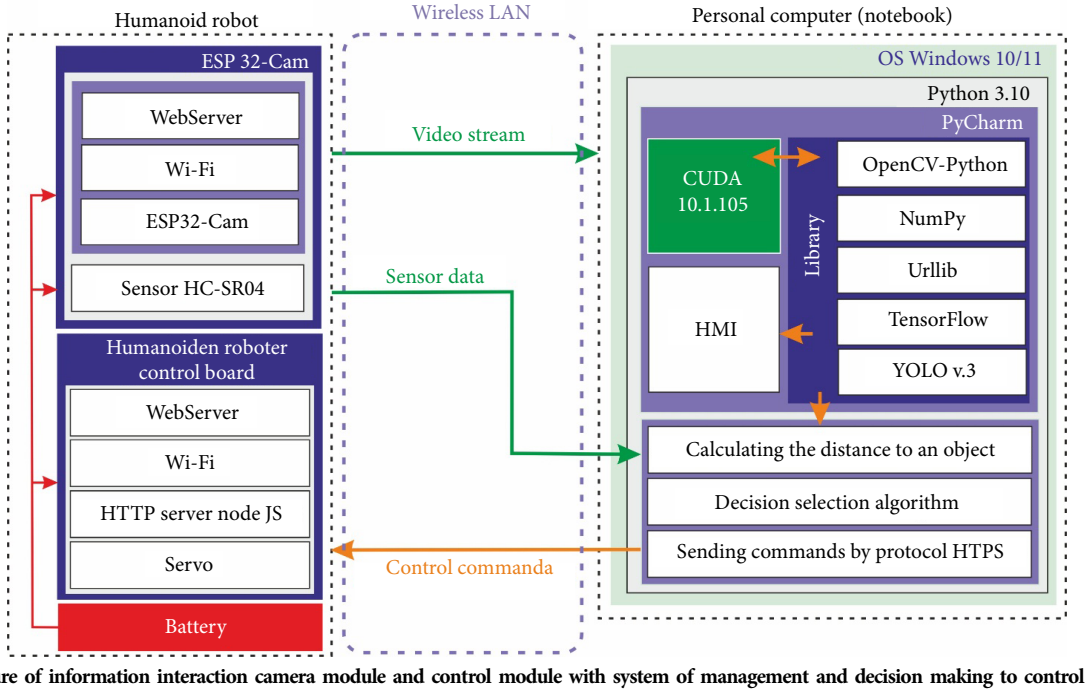


Fig. 4 Structure of information interaction camera module and control module with system of management and decision making to control a humanoid robot.

interact with the ESP32-CAM[29].

Wi-Fi will allow to connect to an existing local network, using authentication tools, via SSID and password.

ESP32-Cam is a driver library for working with the OV2640 camera for the ESP32-Cam hardware module.

For control based on the humanoiden roboter control board hardware module, it is proposed to use the following libraries:

WebServer and Wi-Fi will perform similar functions as on a computer vision system.

HTTP server node JS is a library that makes it possible to receive and process commands to control a humanoid robot from a personal computer.

Servo is a library that allows to control the rotation angles of Emax ES08MA II servomotors.

Battery is modular assembly of batteries with minimum parameters: 5 V and 3000 mA.

HC-SR04 is a hardware module that is an ultrasonic sensor for calculating the distance to an object in the working area of a computer vision system[30].

On the side of a personal computer (notebook), the humanoid robot control and decision-making system is implemented based on the Python 3.10 language[31] in the PyCharm development environment[32] using the following libraries:

OpenCV is a computer vision library that is designed to analyze, classify, and process the received image from ESP32-Cam.

NumPy is a library for processing large multidimensional arrays and matrices with the ability to structure them.

Urllib, HTTP client support library, makes it possible to obtain a URL address for transmitting a video stream from the ESP32-Cam.

TensorFlow is a machine learning library designed to solve the problems of building and training a neural network in order to automatically find and classify patterns found in the working environment of a humanoid robot.

YOLO v.3 is a neural network library (script) for object recognition and identification.

CUDA 10.1.105 is Nvideo's technology library that uses the GPU to accelerate parallel computing in object recognition and identification. In the developed computer vision system, the choice of the CUDA 10.1.105 library is due to the following reasons: parallel computing on the GPU and convenience and compatibility with machine learning and computer vision frameworks such as TensorFlow, PyTorch, and OpenCV. The use of CUDA allows to parallelize the execution of algorithms and data processing, which can lead to a significant acceleration of the process.

A feature of the ESP32 microcontroller is the use of a special ESPAsyncWebServer library, which provides the functionality of a web server, and it supports one access point, that is, one operator. Based on this limitation, one point transmits the streaming video to the computer vision system, and the second one, which is implemented on the humanoiden roboter control board, controls the robot.

It is also necessary to implement the computational functions "Calculating the Distance to an Object", which will allow calculating the distance to the detected object based on the ultrasonic method, and the "Decision Selection Algorithm" block contains behavioral actions, depending on the name of the identified object, distance, and objective functions and tasks of their interaction.

The results of the selected interaction with the object are transferred to the "Sending commands by protocol HTPS" block, in which a sequence of commands is formed and transmitted via the HTPS using the Post and Get methods to the humanoid robot control board hardware module.

For the convenience of visualization and understanding the type of signal transmitted to the circuit, it was decided to designate them in the following colors: green—digital signal for data transmission; black—power circuit "ground" or "gnd"; red—power supply + 5V or "vcc". The electrical circuit diagram for connecting the ESP32-Cam module using the HC-SR04 ultrasonic sensor is shown in Fig. 5.

The assembled test layout of the computer vision system with the HC-SR04 ultrasonic sensor connected is shown in Fig. 6a front view and Fig. 6b back view. Figure 6c is a front view of humanoid robot body; and Fig. 6d is the location of the modules inside the case.

4 Development of Computer Vision System Algorithm for Mobile Humanoid Robot and Its Software Implementation

The next step in the development of a computer vision system for a humanoid robot is the development of an algorithm and a set of

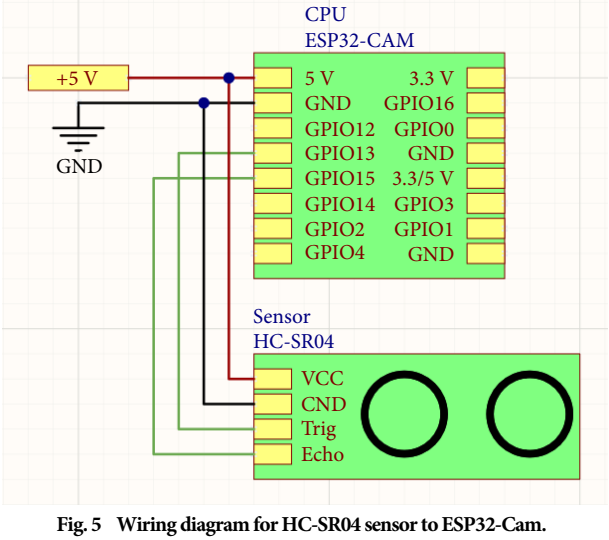
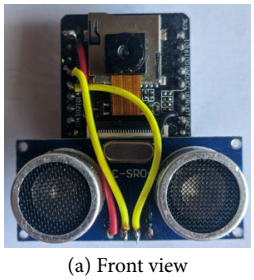
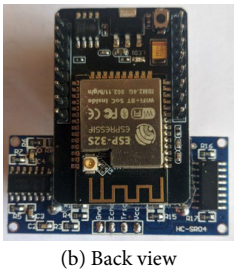


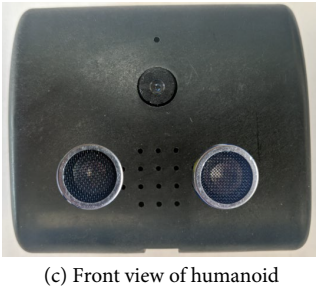
Fig. 5 Wiring diagram for HC-SR04 sensor to ESP32-Cam.



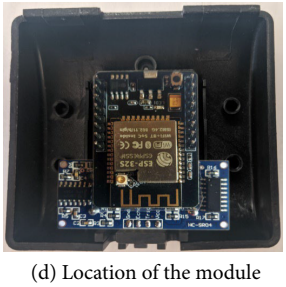
(a) Front view



(b) Back view



(c) Front view of humanoid robot body



(d) Location of the module inside the case

Fig. 6 Placement of a computer vision system in a humanoid robot.

programs for interaction between the ESP32-Cam microcontroller and the HC-SR04 sensor on one hand and for system of management and decision making (notebook) on the other hand. An enlarged algorithm for the operation of a computer vision system is shown in Fig. 7.

Let us briefly describe the purpose of the main blocks of the algorithm (Fig. 7), which are implemented on the basis of the ESP32-Cam hardware module using the HC-SR04 sensor (Fig. 6). When power is connected to the ESP32-Cam module, the libraries are initialized (Fig. 4), after which the local wireless network is searched using authentication data (SSID and password). If the connection fails, then an error message is displayed in the port monitor (Arduino IDE during settings), and upon successful connection, the local network access point (router) allocates a local dynamic IP address of the ESP32-Cam. The video stream and data from the sensor will be transmitted to the system of management and decision making (notebook) at this address. After receiving the local IP address, we activate the 2Mp camera OV2640. Taking into account the peculiarities of the operation of microcontrollers, until the power is turned off on the ESP32-Cam, the video stream from the OV2640 camera, as well as the results of calculating the distance values from the ultrasonic sensor HC-SR04, will be continuously transmitted through the received IP address in a cycle. It is worth noting that all calculations of the distance to the object using the sensor will be carried out on the basis of the ESP32-Cam module, and the calculation results in centimeters will be transmitted to the system of management and decision making.

The operation algorithm of the system of management and decision making (notebook) is built according to the following principle, after starting the program, at the first stage, the libraries are initialized (Fig. 4). After that, it checks for the presence of an IP address assigned to the ESP32-Cam in the local network. If the given IP address is not found, the user is given an error message. If the connection is successful, the program starts receiving the video stream and distance calculation results from the HC-SR04 sensor. The next step of the program is to solve the problem of recognizing the presence of an object in the working area of a humanoid robot. If an object is in the working area, it is identified with the display of the name of the object, the value of the probability of matching and recognition time, as well as the distance to it. Based on the received data (object name and distance), it is possible to implement a decision making mechanism to interact with the object in accordance with the tasks, as a result, the humanoid robot control board module receives commands for execution. At the same time, in real time, all the necessary information (name of the object, probability of coincidence, and distance to it) is visualized to the user in the Human-Machine Interface (HMI) window. The program will function until the user completes work with it or the power is

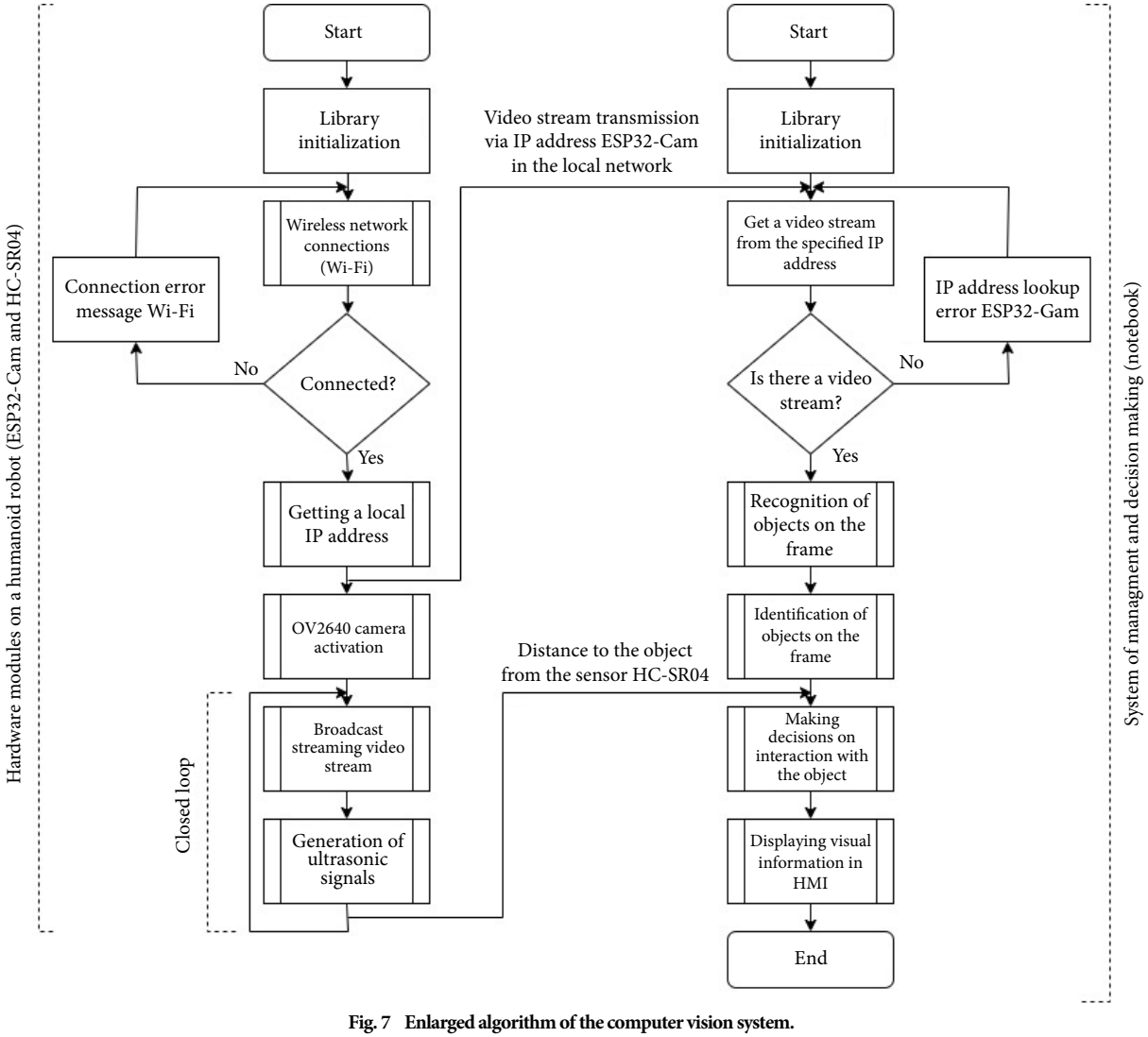


Fig. 7 Enlarged algorithm of the computer vision system.

turned off on the ESP32-Cam module.

When creating a program for the ESP32-Cam module, we will use the Arduino IDE 1.8.19 development environment with the obligatory connection of the following libraries in the boards manager: Arduino AVR Boards ver. 1.8.5 and ESP32 ver.2.0.4 by Espressif System.

At the first step, write a program for broadcasting a video stream from ESP32-Cam, connecting libraries:

#include ⟨WebServer.h⟩;
#include ⟨WiFi.h⟩;
#include ⟨esp32cam.h⟩.

Let us write the constants that will contain the login and password for ESP32-Cam authentication in the local network (set by the developer):

const char* WIFI_SSID = "Crow";
const char* WIFI_PASS = "notebookpuk100".

Let us write the port for connecting to the web server.

WebServer server(80);

We indicate the pins to which the HC-SR04 sensor is connected to the ESP32-Cam:

#define trigPin 13;
#define echoPin 15.

For the convenience of computer vision system testing with different types of resolution obtained from a 2Mp OV2640 camera, we indicate the following working resolutions:

static auto loRes = esp32cam::Resolution::find(320, 240);
static auto midRes = esp32cam::Resolution::find(350, 530);
static auto ords = esp32cam::Resolution::find(800, 600).

That is, it will make it possible, when testing a computer vision system, to change the image quality in the program on the system of management and decision making side. In the future, it will be possible to select only one quality of streaming video for the tasks, which will make it possible to reduce the load on the microcontroller.

Let us create a serveJpg() function to receive a streaming video as a jpg sequence and an error handler in the form of a 503 code for the server[33].

{auto frame = esp32cam::capture();
if (frame == nullptr) {
Serial.println(«CAPTURE FAIL»);
server.send(503, «», «»); // means that right now the server is not ready to process the request, because it is overloaded or technical work is being carried out on it
Return;}
Serial.printf("CAPTURE OK %dx%d %db\n", frame->getWidth(), frame->getHeight(),
static_cast⟨int⟩(frame->size()));
server.setContentLength(frame->size());
server.send(200, "image/jpeg"); // successful request, server is running, information is being transmitted.
WiFiClient client = server.client();
frame->writeTo(client);}

Let us create the functions void handleJpgLo(), void handleJpgHi(), and void handleJpgMid() to check the possibility of streaming video at three resolutions: loRes, midRes, and hires. An example of such an implementation of the void handleJpgLo() function is presented below:

{if (!esp32cam::Camera.changeResolution(loRes)) {
Serial.println("SET-LO-RES FAIL");}
serveJpg();}

It is similarly implemented for the functions void handleJpgHi() and void handleJpgMid().

The next step is to describe the necessary data in the void setup() function, skip the description of the standard port settings, and present the most interesting points:

Set the signal levels on the pins to which the HC-SR04 sensor is connected:

pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);

Set up the OV2640 camera parameters and check its performance:

{using namespace esp32cam;
Config cfg;
cfg.setPins(pins::AiThinker);
cfg.setResolution(ords);
cfg.setBufferCount(2);
cfg.setJpeg(80);
bool ok = Camera.begin(cfg);
Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL"); }

It is worth paying attention to the last line, which allows to display technical information in the port monitor in the Arduino IDE development environment when it is configured.

Connect to a local wireless network using the data entered in WIFI_SSID and WIFI_PASS:

WiFi.persistent(false);
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED).

Upon successful connection, the received local IP address of the ESP32-Cam module is displayed in the Arduino IDE port monitor, which will be needed to broadcast the video stream in system of management and decision making:

Serial.print(«http://»);
Serial.println(WiFi.localIP()).

Let us activate three servers, with the help of which streaming video is transmitted in the form of a sequence of jpg files with different resolutions and turn on the server:

server.on("/cam-lo.jpg", handleJpgLo);
server.on("/cam-hi.jpg", handleJpgHi);
server.on("/cam-mid.jpg", handleJpgMid);
server.begin().

The last step is to write the void loop(), which is executed in a cyclic mode and is a feature of working with microcontrollers:

Prescribe the program code for the operation and calculation of the distance for the HC-SR04 sensor:

{digitalWrite(trigPin, LOW); // we apply a low level signal to pin 13;
delay(200); // make a delay of 200 ms;
digitalWrite(trigPin, HIGH); // we apply a high level signal to pin 13;

This allows to create a sequence of ultrasonic sequences on trigPin, which will then be captured as a reflected signal from objects on the echoPin;

delay(200);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
distance = (duration/2)/29.1; // distance calculation (in centimeters)
Serial.print(distance);
Serial.println(«cm») // information output to the port monitor (for testing)
server.handleClient();}//launch Web Server

Now it is necessary to develop software for recognition and identification of objects and its visualization in the system of management and decision making block. To do this, we will use the PyCharm[33] development environment and the Python 3.10

programming language[32].

In the first step, we include the following libraries:

```
import cv2
import numpy as np
import urllib.request
import matplotlib.pyplot as plt
import time
import sys
import os
```

Set the following weight configuration for object detection:

```
CONF = 0.6
SCORE = 0.3
IOU = 0.7
```

Object detection weights such as Confidence Threshold (CONF), Evaluation Threshold (SCORE), and Overlap Threshold (IOU) are typically chosen based on the experience and requirements of a particular computer vision task. The definition of these weights depends on the specific task and can be selected based on the requirements for the accuracy and completeness of object detection. Optimal values can be determined by experimenting and tuning the model on the training dataset.

Write the path to the neural network configuration file:

```
config = "D:\Yv3\darknetmaster\cfg\yolov3.cfg"
```

Connect the neural network weight file:

```
weights = "D:\Yv3\darknetmaster\yolov3.weights"
font_scale = 1
thickness = 1
```

Connect the library of all object labels:

```
labels =
open("D:\Yv3\darknetmaster\data\coco.names").read().strip().split("\n")
```

Write a color generator for each recognized object:

```
colors = np.random.randint(0, 255, size=(len(labels), 3), dtype="uint8")
```

Load the neural network YOLO:

```
net = cv2.dnn.readNetFromDarknet(config, weights)
```

The next step is to connect to receive a fragment of streaming video in the form of a sequence of jpg files from the ESP32-Cam module, with the following lines:

```
url = http://192.168.1.12/cam-hi.jpg // local IP address of the ESP32-Cam module on the network
cam = cv2.VideoCapture(url)
```

We start working with layers in accordance with the technology incorporated in the YOLO library:

Iterate over all detected objects;

For detection in output

We extract the class identifier (label) and the reliability (as a probability) of detecting the current object;

```
scr = detection[5:]
class_id = np.argmax(scr)
confidence = scr[class_id]
```

Determine the class of the object and evaluate the reliability of the found object;

If confidence > CONF:

Scale the coordinates of the bounding box relative to the size of the image, given that YOLO actually returns the center coordinates (x, y) of the bounding box, followed by the width and height of the boxes;

```
box = detection[:4] * np.array([w, h, w, h])
(centerX, centerY, width, height) = box.astype("int")
```

Use center coordinates (x, y) to get vertex and left corner of bounding box;

```
x = int(centerX – (width / 2)).
y = int(centerY – (height / 2)).
```

Update our list of bounding box coordinates, confidences, and class ids;

```
boxes.append([x, y, int(width), int(height)])
confidences.append(float(confidence))
class_ids.append(class_id)
```

We start working with detected objects, provided that there is at least one:

Iterate over the stored indexes;

For i in id.flatten():

Extract the coordinates of the bounding box;

```
x, y = boxes[i][0], boxes[i][1]
w, h = boxes[i][2], boxes[i][3]
```

Draw a bounding box rectangle and label the image;

```
color = [int I for c in colors[class_ids[i]]]
cv2.rectangle(im, (x, y), (x+w, y+h), color=color, thickness=thickness)
text = f"{labels[class_ids[i]]}: {confidences[i]:.2f}"
```

Calculate the width and height of the text to draw transparent fields as the background of the text;

```
(text_width, text_height) = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, fontScale=font_scale, thickness=thickness)[0]
text_offset_x = x
text_offset_y = y – 5
box_coords = ((text_offset_x, text_offset_y), (text_offset_i + text_width + 2, text_offset_y – text_height))
overlay = im.copy()
cv2.rectangle(overlay, box_coords[0], box_coords[1], color=color, thickness=cv2.FILLED)
- add opacity (field transparency);
image = cv2.addWeighted(overlay, 0.6, im, 0.4, 0)
- now place the text (label: trust (%));
cv2.putText(im, text, (x, y–5), cv2.FONT_HERSHEY_SIMPLEX,
fontScale=font_scale, color=(0, 0, 0), thickness=thickness)
cv2.imshow("Detection", im)
if ord("q") == cv2.waitKey(5):
```

If you need to complete the work with the program, add the HMI form close handler:

```
cam.release()
cv2.destroyAllWindows()
```

We have to pay attention to the fact that, depending on the tasks set, the user can rewrite the neural network configuration file, as well as the weights file. Depending on the number of stored object labels, it makes it possible to increase the number of simultaneously identified objects in the working area of a humanoid robot, and the developer can create his own label library, which will increase the quality of identification and reduce time.

To successfully flash the microcontroller based on the ESP32-Cam module, we need to select the following configuration setting in the Arduino IDE development environment[28] in the Tools menu, as shown in Fig. 8.

After successful installation of the required firmware, the ESP32-Cam should be restarted through the tools menu of the serial monitor, which includes mainly setting the dynamic IP address of the module in the local network, the Joint Photographic Expert Group (jpg) sequence and resolution, and other features. An example of information displayed in the serial monitor is shown in Fig. 9.

An example of the HMI of the developed computer vision system for a humanoid robot is shown in Fig. 10.

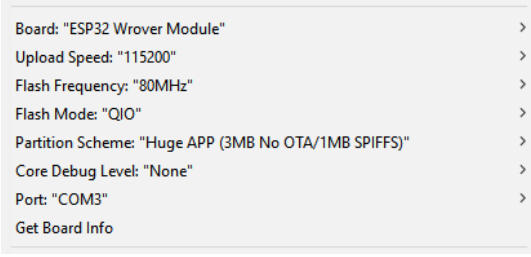


Fig. 8 Arduino IDE Tools menu settings for ESP32-Cam firmware.

As you can see in Fig. 10, a standard set of configuration files and label files was used, and the developed computer vision system for a humanoid robot identifies objects. Now you can develop your own library assembly to solve the identification of the "ball" object, and in the future this library can be expanded depending on requirements.

5 Experiment

Let us conduct a series of natural experiments with the developed computer vision system for a humanoid robot to assess its performance. To do this, we will assemble a layout that will simulate the location of the computer vision system inside a humanoid robot, taking into account the vertical height of its location from the floor, which will make it possible to check the performance of the HC-SR04 sensor in real conditions of its operation. An example of a layout for conducting experiments is shown in Fig. 11.

At the first stage, the research on the recognition and identification of a statistical object of the form of a ball with the label "robo ball" is conducted, and in this case, the classic coco.names library is used. To do this, we place a static object "robo ball" at different distances from the computer vision system and measure the following characteristics: the probability of coincidence and recognition time, under the general condition that the illumination is 497 Lux (standard office and natural). The results of the first experiment are shown in Table 4.

An obtained result can be explained by the peculiarity of the architecture of the Convolutional Neural Network (CNN). In this case, when recognizing an object, "robo ball" is applied once to the entire image at once. The network divides the image into a kind of grid and predicts the bounding boxes and the probability that there is the desired object for each area, which affects the large spread, where the range of object identification accuracy changes from 0.37 (for objects that strongly affect the accuracy) to 0.94 (for objects with have a minor effect). From this we can conclude that the use of CNN to solve the problem is not effective, due to the high divergence of identification accuracy at different distances from a mobile humanoid robot. As a result, there is no point in conducting an experiment for the dynamic object "robo ball" with this library.

We will improve the neural network, due to the idea of improvement, which is that S-SSD[34] replaces the VGG-16 network[35] as the basic network for extracting features of the Single Shot Detector (SSD) network model with a tiny-YOLO lightweight network model[36]. This will make it possible to preserve the multidimensional structure of the network model feature graph, which will increase the detection accuracy, while significantly reducing the amount of calculations generated by the network operation, thereby significantly increasing the detection speed, due to a limited set of objects (in the "robo ball" experiment). It will also remove time delays when working in a dynamic environment. Let us repeat the conditions of the first experiment based on the tiny-YOLO lightweight network model for recognizing and identifying the "robo ball" object in room conditions. The results of the second experiment are presented in Table 5.

For the convenience of analyzing the data obtained in the two experiments described above, we present them in the form of a dependence graph: recognition accuracy and time on distance, for the classic coco.names library and the lightweight tiny-YOLO network model. The resulting graphs are shown in Figs. 12 and 13, respectively.

As can be seen from Fig. 12, the computer vision system based on the coco.names standard library for the "robo ball" object has an identification accuracy of 0.59–0.94, while the identification

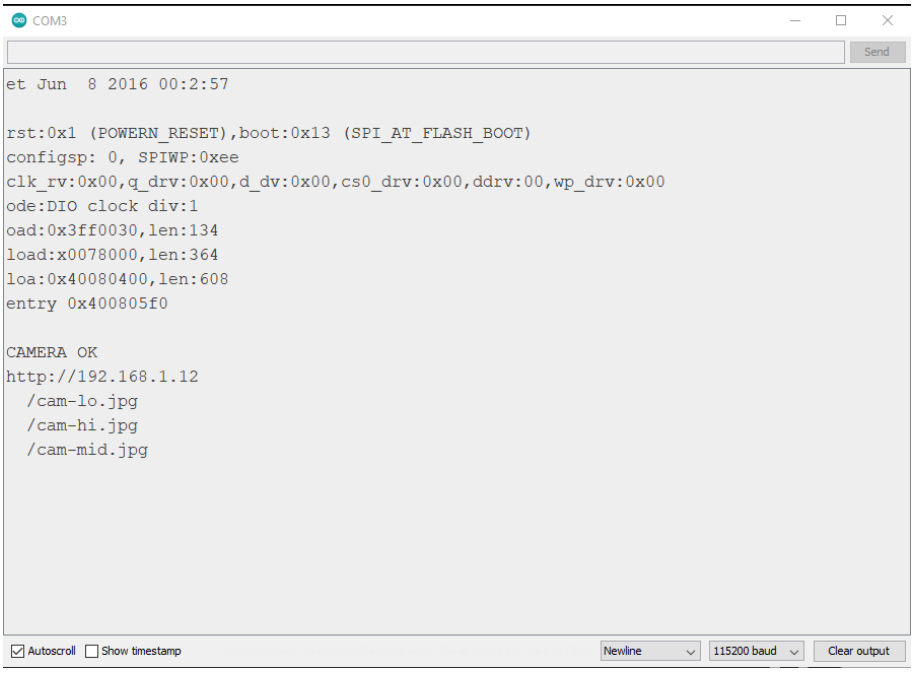


Fig. 9 Example of technical information about a successful ESP32-Cam firmware.

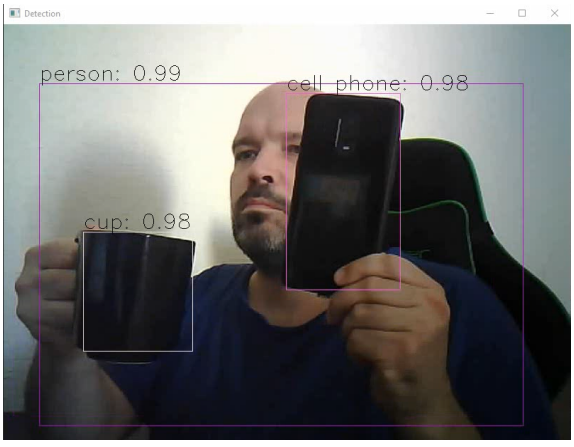
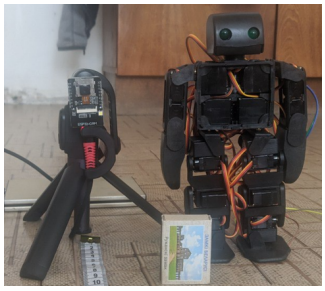


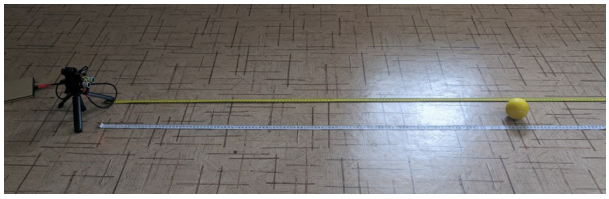
Fig. 10 HMI of a developed computer vision system for a humanoid robot.



(a) Calibrating the location of the computer vision system for the test



(b) General view of the computer vision system



(c) Experimental layout for testing

Fig. 11 Experimental layout for research.

Table 4 Results of the first experiment.

Distance (cm)	Obtained result	Accuracy	Distance from sensor HC-SR04 (cm)	Identification time (s)
35		0.87	35	0.382
50		0.59	51	0.397
100		0.87	103	0.373
150		0.94	156	0.41
200		0.60	206	0.38

Table 5 Results of the second experiment.

Distance (cm)	Obtained result	Accuracy	Distance from sensor HC-SR04	Identification time (s)
35		0.98	35	0.024
50		0.99	51	0.028
100		0.97	103	0.026
150		0.94	156	0.032
200		0.80	206	0.031

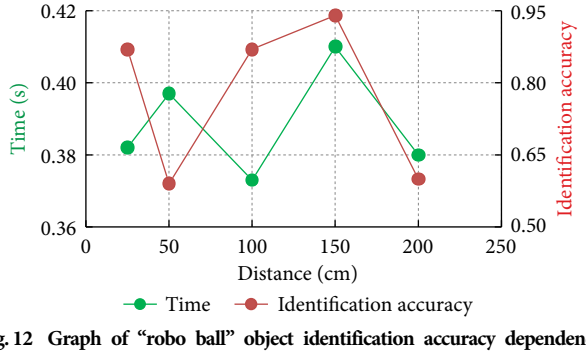


Fig. 12 Graph of "robo ball" object identification accuracy dependence, time from distance for the classic library coco.names.

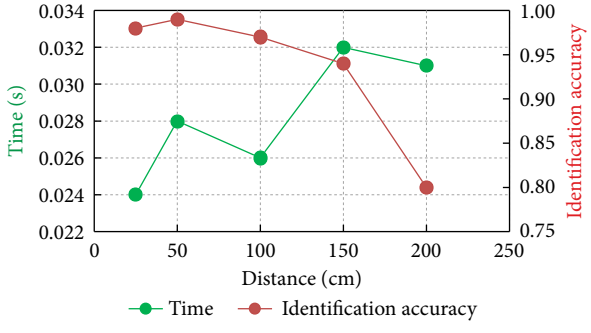


Fig. 13 Graph of "robo ball" object identification accuracy dependence, time from distance for a tiny-YOLO lightweight network model.

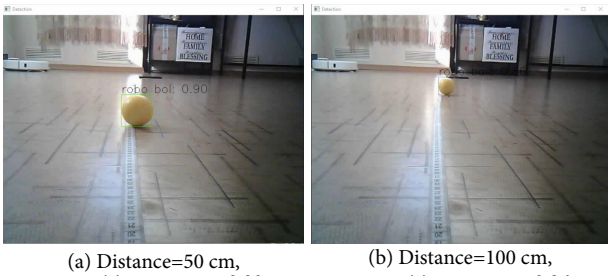
time is in the intervals from 0.373–0.410 s and the graph has a difficultly unpredictable shape that does not depend on distance to the object. In contrast to the graph of the tiny-YOLO lightweight network model (Fig. 13), it can be seen that the identification accuracy depends on the distance, that is, the farther the "robo ball" object is from the humanoid robot, the probability of belonging to this label decreases, while it is worth noting that to solve the problem in this study, the distance of 200 cm is large enough and will not be used.

The created dataset for the task of detecting and recognizing objects of a mobile humanoid robot is built on the basis of the tiny-YOLO lightweight network model, surpasses the SSD network model in terms of detection accuracy and detection speed, and can simultaneously meet the requirements of detection accuracy

and real-time performance (in dynamics) when performing indoor tasks. These conclusions are confirmed by comparative experiments on the accuracy and speed of object detection, as well as on the detection and recognition of objects in real time by a mobile robot in a real indoor environment.

The next experiment is the recognition and identification of an object with the label "robo ball" in dynamics, that is, the essence of the experiment is to identify a dynamically receding object, and in fact, this is a simulation of a humanoid robot playing football. The results of such an experiment are shown in Fig. 14.

Analyzing the obtained results of identifying the "robo ball" object in dynamics, it can be seen that the recognition accuracy of the improved neural network at a distance of 100 cm and 150 cm (Figs. 14b and 14c) was affected by the presence of the object in direct sunlight, which gave the appearance of lightening of the contours of the object , and as a result, it led to a decrease in the recognition accuracy, in contrast to the results obtained at distances of 200 cm (Fig. 14d), in which the object left the clarification zone, which accordingly increased the recognition accuracy to 0.97. Despite the obtained results, it can be concluded that the neural network improvements based on the tiny-YOLO lightweight network model provide high speed and accuracy of object recognition with a low camera resolution on the ESP32-Cam (2Mp) module for small-sized mobile humanoid robots.



(a) Distance=50 cm, recognition accuracy 0.90

(b) Distance=100 cm, recognition accuracy 0.86

(c) Distance=150 cm, recognition accuracy 0.86

(d) Distance=200 cm, recognition accuracy 0.97

Fig. 14 Identification of the "robo ball" object in dynamics (distance increasing).

Let us compare the developed computer vision system with a control system for a humanoid robot, with similar solutions. The comparison results are presented in Table 6.

Let us build Table 7 to compare the identification time of the "ball" object from distance for the developed computer vision system based on ESP32-Cam with a computer vision system based on NVIDIA Jetson Nano[38]

Let us present the comparison results obtained from Table 7 in the form of a graph, which is shown in Fig. 15.

6 Conclusion

The developed computer vision system for small-sized mobile humanoid robots was implemented on the basis of ESP32-Cam using an ultrasonic sensor HC-SR0. To speed up the process of recognition and identification of the "robo ball" object, the authors proposed a tiny-YOLO lightweight network model. This solution allowed us to preserve the multidimensional structure of the feature graph of the network model, and therefore increase the detection accuracy, while significantly reducing the amount of calculations generated by the network operation, thereby significantly increasing the detection speed when using microcontroller systems. As can be seen from the results of the experiments, the improved neural network made it possible to obtain an average time for identifying the object "robo ball" 0.028 in natural conditions, with a high probability from 0.99 at a distance of 50 cm to 0.80 at 200 cm, while it is worth noting that a camera is used OV2640 2MP/FOV70. In the future, it is planned to develop a target detection system and implement mechanisms for interacting with "objects", as well as consider the possibility of implementing movement trajectory prediction to predict actions.

The experimental results obtained show that the developed computer vision system has a number of significant advantages,

Table 6 Comparison of the developed computer vision system with a control system for a humanoid robot with similar solutions.

Parameter	Similar solution for mobile humanoid robots		
	Baihaqi et al.[37]	Nugraha et al.[38]	Computer vision system developed
Microcontroller	ESP32-CAM	NVIDIA Jetson Nano	ESP32-CAM
Operating frequency	160 MHz	1.43 GHz	160 MHz
Camera	OV2640 2Mp/FOV70	Logitech C525 8 MP	OV2640 2Mp/FOV70
Camera resolution in the process of identification (pixel)	GIF (320×240)	HD (1280×720)	SVGA (350×530)
Type of underlying neural network	– (Photo storage)	YOLOv3	Improved neural network
Object type	Human face	Ball (ideal conditions*)	Ball (not ideal conditions**)
Average recognition time	Without mask 1.26c With mask 7.24c	0.033	0.028
Success probability (%)	80	Distance=50 cm: 99 Distance=900 cm: 46	Distance=50 cm: 99 Distance=200 cm: 80
Dimension (mm×mm×mm)	27×39×4.5	100×80×29	27×39×4.5
System price (US dollar)	8	180	8

Note: *: The studies were carried out under ideal conditions in terms of the contrast of choosing the color of the identified object "ball" ("yellow" RGB (223, 97, 10)) and the background ("green" RGB (55, 183, 168)). **: The studies were carried out in a natural environment, the color of the identified object "ball" ("yellow" RGB (173, 145, 97)) background in the form of linoleum (RGB (197, 192, 189)).

Table 7 Comparison of the ball object identification time versus distance for the developed computer vision system based on ESP32-Cam with the computer vision system based on NVIDIA Jetson Nano[38].

Compared system	Identification time (s)			
	Distance= 50 cm	Distance= 100 cm	Distance= 200 cm	Distance= 300 cm
Computer vision system based on NVIDIA Jetson Nano[38]	0.033	0.032	0.033	0.033
Developed computer vision system	0.028	0.026	0.031	–

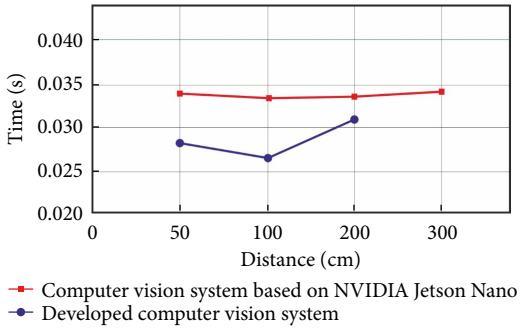


Fig. 15 Comparison of the ball object identification time versus distance for the developed computer vision system with a similar solution based on NVIDIA Jetson Nano[38].

such as high recognition speed comparable to the expensive NVIDIA Jetson Nano and improved AI YOLO v.3 algorithm implemented for ESP32-Cam, which gives a minimal identification error for a dynamic object and has minimal requirements for the power module, moreover, the price of the developed computer vision module does not exceed 8 US dollar, compared to analogues of 180 US dollar. However, the developed solution is not without drawbacks. Restrictions on overall dimensions have significantly reduced the computing power of the module, as a result of which the solution of some recognition and identification problems must be solved on a separate device. Thus, this problem and its solution is the next direction of possible research.

Dates

Received: 1 May 2023; Revised: 15 September 2023; Accepted: 18 September 2023

References

[1] S. Khlamov, V. Savanevych, I. Tabakova, and T. Trunova, Statistical modeling for the near-zero apparent motion detection of objects in series of images from data stream, in *Proc. 12th Int. Conf. Advanced Computer Information Technologies* (*ACIT*), Ruzomberok, Slovakia, 2022, pp. 126–129.

[2] S. Khlamov, I. Tabakova, and T. Trunova, Recognition of the astronomical images using the Sobel filter, in *Proc. 29th Int. Conf. Systems, Signals and Image Processing* (*IWSSIP*), Sofia, Bulgaria, 2022, pp. 1–4.

[3] M. A. Ahmad, I. Tvoroshenko, J. H. Baker, and V. Lyashenko, Modeling the structure of intellectual means of decision-making using a system-oriented NFO approach, *Int. J. Emerg. Trends Eng. Res.*, vol. 7, no. 11, pp. 460–465, 2019.

[4] V. Lyashenko, O. Kobylin, and M. Minenko, Tools for investigating the phishing attacks dynamics, in *Proc. Int. Scientific-Practical Conf. Problems of Infocommunications. Science and Technology* (*PIC S&T*), Kharkiv, Ukraine, 2018, pp. 43–46.

[5] H. Attar, A. T. Abu-Jassar, V. Yevsieiev, V. Lyashenko, I. Nevliudov, and A. K. Luhach, Zoomorphic mobile robot development for vertical movement based on the geometrical family caterpillar, *Comput. Intell. Neurosci.*, vol. 2022, p. 3046116, 2022.

[6] H. Attar, A. T. Abu-Jassar, A. Amer, V. Lyashenko, V. Yevsieiev, and M. R. Khosravi, Control system development and implementation of a CNC laser engraver for environmental use with remote imaging, *Comput. Intell. Neurosci.*, vol. 2022, p. 9140156, 2022.

[7] A. Rabotiahov, O. Kobylin, Z. Dudar, and V. Lyashenko, Bionic image segmentation of cytology samples method, in *Proc. 14th Int. Conf. Advanced Trends in Radioelecrtronics, Telecommunications and Computer Engineering* (*TCSET*), Lviv-Slavske, Ukraine, 2018, pp. 665–670.

[8] S. M. H. Mousavi, V. Lyashenko, and V. B. S. Prasath, Analysis of a robust edge detection system in different color spaces using color and depth images, *Comput. Opt.*, vol. 43, no. 4, pp. 632–646, 2019.

[9] A. T. Abu-Jassar, Y. M. Al-Sharo, V. Lyashenko, and S. Sotnik, Some features of classifiers implementation for object recognition in specialized computer systems, *TEM J.*, vol. 10, no. 4, pp. 1645–1654, 2021.

[10] A. C. Bavelos, N. Kousi, C. Gkournelos, K. Lotsaris, S. Aivaliotis, G. Michalos, and S. Makris, Enabling flexibility in manufacturing by integrating shopfloor and process perception for mobile robot workers, *Appl. Sci.*, vol. 11, no. 9, p. 3985, 2021.

[11] A. S. M. Al-Obaidi, A. Al-Qassar, A. R. Nasser, A. Alkhayyat, A. J. Humaidi, and I. K. Ibraheem, Embedded design and implementation of mobile robot for surveillance applications, *Indonesian J. Sci. Technol.*, vol. 6, no. 2, pp. 427–440, 2021.

[12] Y. H. Jung, D. H. Cho, J. W. Hong, S. H. Han, S. B. Cho, D. Y. Shin, E. T. Lim, and S. S. Kim, Development of multi-sensor module mounted mobile robot for disaster field investigation, *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. XLIII-B3-2022, pp. 1103–1108, 2022.

[13] G. Lajkó, R. N. Elek, and T. Haidegger, Endoscopic image-based skill assessment in robot-assisted minimally invasive surgery, *Sensors*, vol. 21, no. 16, p. 5412, 2021.

[14] M. M. Rahman, T. Rahman, D. Kim, and M. A. U. Alam, Knowledge transfer across imaging modalities via simultaneous learning of adaptive autoencoders for high-fidelity mobile robot vision, in *Proc. 2021 IEEE/RSJ Int. Conf. Intelligent Robots and Systems* (*IROS*), Prague, Czech Republic, 2021, pp. 1267–1273.

[15] Y. J. Mon, Vision robot path control based on artificial intelligence image classification and sustainable ultrasonic signal transformation technology, *Sustainability*, vol. 14, no. 9, p. 5335, 2022.

[16] D. Zhang and Z. Guo, Mobile sentry robot for laboratory safety inspection based on machine vision and infrared thermal imaging detection, *Secur. Commun. Netw.*, vol. 2021, p. 6612438, 2021.

[17] A. Stateczny, K. Gierlowski, and M. Hoeft, Wireless local area network technologies as communication solutions for unmanned surface vehicles, *Sensors*, vol. 22, no. 2, p. 655, 2022.

[18] EMAX ES08MA II 12g mini metal gear analog servo for RC model, https://www.amazon.com/ES08MA-Metal-Analog-Servo-Model/dp/B07KYK9N1G, 2023.

[19] OV7670 camera module supports VGA CIF auto exposure control display active size 640X480, https://www.amazon.com/Supports-Exposure-Control-Display-640X480/dp/B09X59J8N9/ref=sr_1_6?crid=1OSVL1M09YPB5&keywords=VGA+OV7670&qid=1669977970&sprefix=vga+ov7670+%2Caps%2C600&sr=8-6, 2023.

[20] 01Studio pyai-K210 kit entwicklungs board python ai künstliche intelligenz machine vision deep learning micro python, https://de.aliexpress.com/item/1005001459205624.html?gatewayAdapt=glo2deu, 2023.

[21] ESP32-CAM esp32 cam seriell zu wifi entwicklung bord micro usb bluetooth + ov2640 kamera modul, https://de.aliexpress.com/item/32947577882.html?gatewayAdapt=glo2deu, 2023.

[22] Preliminary datasheet, OV7670/OV7171 CMOS VGA (640×480), http://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf, 2023.

[23] M5Stack K027 M5StickV K210 ai camera, https://eu.mouser.com/

new/m5stack/m5stack-k027-m5stickv-k210-ai-camera/, 2023.
[24] ESP32-Cam module product specification, https://loboris.eu/ESP32/ESP32-CAM%20Product%20Specification.pdf, 2023.
[25] Neue version 32 kanal roboter servo control board servo motor controller PS2 drahtlose steuerung USB/UART verbindung modus, https://de.aliexpress.com/item/32272674301.html?gatewayAdapt=glo2deu, 2023.
[26] straße lenkgetriebe control board servo controller von intelligente roboter/serielle USB/PC APP/motor, https://de.aliexpress.com/item/1005004170128663.html?gatewayAdapt=glo2deu, 2023.
[27] Plen2 control board entwicklung board wireless controller für 18DOF vivi RC biped humanoiden roboter DIY für arduino, https://de.aliexpress.com/item/4000432667382.html?gatewayAdapt=glo2deu, 2023.
[28] Arduino IDE, https://www.arduino.cc/en/software, 2023.
[29] P. A. Nugroho, Kontrol lampu gedung melalui WiFi esp8266 dengan web server lokal, *J. Elektro Dan Informatika Swadharma*, vol. 1, no. 2, pp. 1–11, 2021.
[30] Ultrasonic module HC-SR04 distance sensor, https://www.amazon.eg/-/en/Ultrasonic-Module-HC-SR04-Distance-arduino/dp/B091D3QWN4, 2023.
[31] Python release python 3.10, https://www.python.org/downloads/release/ python-3100/, 2023.
[32] PyCharm, https://www.jetbrains.com/pycharm/, 2023.
[33] What is a 503 status code? https://www.webfx.com/web-development/glossary/http-status-codes/what-is-a-503-status-code/, 2023.
[34] L. Withington, D. D. P. D. Vera, C. Guest, C. Mancini, and P. Piwek, Artificial neural networks for classifying the time series sensor data generated by medical detection dogs, *Expert Syst. Appl.*, vol. 184, p. 115564, 2021.
[35] Z. Wang, X. Gu, R. S. M. Goh, J. T. Zhou, and T. Luo, Efficient spiking neural networks with radix encoding, *IEEE Trans. Neural Netw. Learning Syst*., doi: 10.1109/TNNLS.2022.3195918.
[36] B. C. Liu, Q. Yu, J. W. Gao, S. Zhao, X. C. Liu, and Y. F. Lu, Spiking neuron networks based energy-efficient object detection for mobile robot, in *Proc. 2021 China Automation Congress* (*CAC*), Beijing, China, 2021, pp. 3224–3229.
[37] M. Y. Baihaqi, V. Vincent, and J. W. Simatupang, Humanoid robot application as COVID-19 symptoms checker using computer vision and multiple sensors, *ELKHA: J. Teknik Elektro*, vol. 13, no. 2, pp. 105–112, 2021.
[38] A. C. Nugraha, M. L. Hakim, S. Yatmono, and M. Khairudin, Development of ball detection system with YOLOv3 in a humanoid soccer robot, *J. Phys*.: *Conf. Ser*., vol. 2111, no. 1, p. 012055, 2021.